

Phantom Fob

Reverse Engineering a Rolling-Code Key Fob

Technical challenge writeup

Fair-play note: the final UNLOCK opcode and challenge flag are intentionally omitted so future players can still complete the final step themselves.

Overview

This challenge presented a simulated vehicle instrument cluster through a web application. The dashboard exposed live vehicle state while a virtual key fob provided LOCK, HORN, IMMOB_ARM, and IMMOB_DISARM controls - but deliberately no UNLOCK button.

- Door lock state
- Immobiliser state
- Horn state
- Vehicle speed and turn signals
- CAN frames per second
- Number of observed CAN IDs
- A flag field released after satisfying the challenge condition

The goal was to determine how the vehicle accepted key-fob commands over CAN and construct a valid UNLOCK command without revealing the final opcode in the published writeup.

1. Initial Web Reconnaissance

The application exposed two obvious HTTP endpoints. The dashboard consumed Server-Sent Events from:

```
GET /events
```

The virtual key-fob controls submitted JSON to:

```
POST /press
Content-Type: application/json

{"button": "LOCK"}
```

Submitting the obvious missing command returned a server-side rejection:

```
{"button": "UNLOCK"}

→ {"msg": "no such control", "ok": false}
```

This established that UNLOCK was not simply a hidden web control. The dashboard HTML also contained several useful clues:

```
Instrument Cluster - live CAN
IDs seen: 12
Flag released on the bus
```

Those hints strongly suggested that the intended solution involved the CAN bus itself rather than a web-only bypass.

2. Discovering socketcand

A TCP port scan revealed three exposed services:

```
22/tcp      SSH
8080/tcp    Werkzeug web application
29536/tcp   unknown
```

Connecting to TCP/29536 returned:

```
< hi >
```

That greeting identified the service as socketcand, which exposes Linux SocketCAN interfaces over TCP. Opening the virtual CAN interface and switching to raw mode produced live traffic:

```
< open vcan0 >
< rawmode >

< ok >
< ok >
< frame 429 1789173655.554768 010000a500000000 >
```

3. Identifying the Vehicle-State Frame

One periodic CAN ID had an unusually simple payload:

```
429  01 00 00 a5 00 00 00 00
```

By pressing legitimate controls and correlating changes, the first three bytes mapped cleanly to door, immobiliser, and horn state:

```
01 00 00 a5 00 00 00 00  LOCKED / DISARMED / HORN OFF
01 01 00 a5 00 00 00 00  LOCKED / ARMED      / HORN OFF
01 00 01 a5 00 00 00 00  LOCKED / DISARMED / HORN ON
```

That made the obvious unlocked-state candidate:

```
00 00 00 a5 00 00 00 00
```

Injecting that state frame caused the dashboard to display UNLOCKED. Repeated injection could keep the UI in that state, but the challenge did not complete.

Key insight: CAN ID 429 represented vehicle status, not the authenticated key-fob command that caused the vehicle state machine to transition.

4. Finding the Key-Fob Command Frame

A correlation script captured CAN activity immediately around each /press request. One relatively quiet CAN ID appeared exactly once per fob action: 12A.

```
LOCK
12A  2c 3c 49 b0 ad dc 48 ef

HORN
12A  34 19 ea b0 f7 a0 48 ef

IMMOB_ARM
12A  3c 3e 89 b0 3f 1c 48 ef

IMMOB_DISARM
12A  40 f4 6e b0 57 95 48 ef
```

Repeated captures revealed that byte 0 advanced monotonically, while byte 1 remained constant for each command. The packet structure initially looked like:

```
Byte 0  Rolling counter
Byte 1  Command opcode
Byte 2  Changing value
Byte 3  Constant b0
Byte 4  Changing value
Byte 5  Changing value
Byte 6  Constant 48
Byte 7  Constant ef
```

5. Replay Protection

A valid HORN frame was captured and retransmitted exactly. The horn did not activate. Changing only the visible counter to the next value also failed. Even after wrapping the visible 8-bit counter through a complete cycle, an old frame remained invalid.

This demonstrated that the receiver enforced freshness beyond the visible low counter byte and that the other changing fields participated in validation.

6. Concurrency Experiment

The decisive breakthrough came from issuing many legitimate /press requests nearly simultaneously. A burst produced frames such as:

```
56 3e 65 b0 01 e8 48 ef
57 3c 65 b0 00 e8 48 ef
```

```
58 f4 65 b0 b9 e8 48 ef
59 19 65 b0 df e8 48 ef
5a f4 65 b0 bb e8 48 ef
```

During the burst, bytes 2 and 5 remained constant while the counter and command changed. Under normal timing both values changed, strongly suggesting they represented short-lived freshness material such as a timestamp-derived value or nonce.

More importantly, byte 4 obeyed a simple additive relationship:

```
Byte4 = (Byte0 + Byte1 + Byte2 + Byte5 + 0x20) & 0xff
```

For example:

```
56 3e 65 b0 01 e8 48 ef

0x56 + 0x3e + 0x65 + 0xe8 + 0x20 = 0x201
0x201 & 0xff = 0x01
```

The concurrency experiment had exposed the validation checksum.

7. Forging a Valid Known Command

Before touching the missing UNLOCK command, the reconstructed packet format was verified against a known action. A fresh legitimate frame supplied the current rolling counter and freshness bytes. A forged IMMOB_ARM packet was then generated using the next counter value and the recovered checksum formula:

```
checksum = (counter + command + freshness1 + freshness2 + 0x20) & 0xff
```

The resulting forged packet followed this layout:

```
[counter]
[IMMOB_ARM]
[freshness1]
[b0]
[checksum]
[freshness2]
[48]
[ef]
```

After injection, the vehicle changed from DISARMED to ARMED without using the legitimate ARM control. This confirmed that the protocol had been reconstructed correctly and fresh valid fob commands could be generated.

8. Locating the Missing UNLOCK Command

At this point only the one-byte UNLOCK opcode remained unknown. Known command-byte values were:

```
HORN          = 0x19
LOCK          = 0x3c
IMMOB_ARM     = 0x3e
IMMOB_DISARM  = 0xf4
UNLOCK        = intentionally omitted
```

Because the field was only one byte, the remaining search space was small. For each candidate, the test harness obtained current freshness material from a legitimate command, advanced to the expected next counter, recalculated the checksum, injected the candidate, and monitored the real vehicle-state frame.

```
for opcode in range(256):
    fresh = capture_legitimate_frame()
    counter = (fresh.counter + 1) & 0xff

    checksum = (
        counter
        + opcode
        + fresh.byte2
        + fresh.byte5
        + 0x20
    ) & 0xff

    forged = [
        counter, opcode, fresh.byte2, 0xb0,
        checksum, fresh.byte5, 0x48, 0xef
    ]

    inject(forged)

    if vehicle_unlocked():
        break
```

One candidate caused the genuine vehicle state machine to transition to UNLOCKED and satisfied the challenge. The final opcode and flag are intentionally withheld.

9. Recovered Protocol Structure

Byte	Meaning	Observed value / role
0	Rolling counter	Monotonic low byte
1	Command opcode	LOCK / HORN / ARM / DISARM / UNLOCK
2	Freshness A	Time/nonce-like

3	Constant	0xb0
4	Checksum	8-bit additive checksum
5	Freshness B	Time/nonce-like
6	Constant	0x48
7	Constant	0xef

Checksum:

$$B4 = (B0 + B1 + B2 + B5 + 0x20) \bmod 256$$

10. Lessons Learned

Several details in the challenge were useful hints rather than decoration. “Key fob - the buttons you have (no unlock)” suggested that UNLOCK existed at the protocol level even though the UI did not expose it. “Flag released on the bus” hinted that changing displayed state alone would not satisfy the challenge.

The critical distinction was: 429 = vehicle state/status, while 12A = key-fob command.

The concurrency experiment was especially valuable because it froze normally changing freshness fields long enough to expose the checksum relationship. What initially resembled a more sophisticated rolling-code authentication system ultimately reduced to freshness material, a rolling counter, a command opcode, and an 8-bit additive checksum.

11. Solution Path

```

Web reconnaissance
  ↓
Discover socketcand
  ↓
Capture raw CAN traffic
  ↓
Identify vehicle-state ID
  ↓
Spoof unlocked status
  ↓
Realize status spoofing is insufficient
  ↓
Identify fob-command ID
  ↓
Decode rolling counter and command byte
  ↓
Demonstrate replay protection
  ↓
Use concurrent legitimate requests

```

```
↓  
Expose freshness behavior  
↓  
Recover checksum algorithm  
↓  
Forge a known valid command  
↓  
Search the remaining command field  
↓  
Trigger genuine UNLOCK behavior  
↓  
Challenge complete
```

Conclusion

This was an excellent challenge combining web reconnaissance, service identification, socketcand / SocketCAN, CAN traffic analysis, state-versus-command differentiation, replay testing, concurrency testing, protocol reverse engineering, checksum recovery, and CAN-frame forgery.

The most satisfying aspect was that the challenge could not be solved merely by altering the visible dashboard state. It required understanding the actual command path and constructing a frame that the simulated receiver accepted as a fresh, legitimate key-fob command.

The final challenge-specific UNLOCK opcode and flag are intentionally omitted so future players can still complete the last step themselves.